

Fast and Fusiest: An Optimal Fusion-Aware Mapper for Accelerator Design

Tanner Andrulis*
andrulis@mit.edu
MIT
Cambridge, MA, USA

Vivienne Sze
sze@mit.edu
MIT
Cambridge, MA, USA

Michael Gilbert*
gilbertm@mit.edu
MIT
Cambridge, MA, USA

Joel S. Emer
emer@csail.mit.edu
MIT / Nvidia
Cambridge, MA, USA

Abstract

A low-latency and energy-efficient tensor algebra accelerator design must optimize how data movement and operations are scheduled (i.e., mapped) in the accelerator architecture. A key mapping optimization is fusion, meaning holding data on-chip between computation steps in the workload, which has been shown to reduce energy and latency by reducing expensive off-chip data movement. However, the optimal fusion choice depends on the workload and workload shape, and a mapper, which searches for the optimal mapping, can improve energy and latency significantly. However, prior mappers cannot find optimal mappings with fusion (i.e., fused mappings) in a feasible runtime because the number of fused mappings to search increases exponentially with the number of computation steps in the workload.

In this paper, we introduce the Fast and Fusiest Mapper (FFM), a mapper to quickly find optimal mappings in a comprehensive fused mapspace for tensor algebra workloads. FFM shrinks the search space by pruning subsets of mappings (i.e., partial mappings) that are shown to never be a part of optimal mappings, quickly eliminating all suboptimal mappings containing those partial mappings. Then FFM joins partial mappings to construct optimal fused mappings. Using FFM, we demonstrate an energy-delay-product (EDP) reduction by up to 1.8 $\times$ compared to TransFusion, a state-of-the-art accelerator with hand-optimized fusion. Moreover, we show that FFM finds mappings orders of magnitude faster ($> 10,000\times$) than prior automated mappers TileFlow and SET, and given the same runtime, reduces EDP by $> 2\times$.

1 Introduction

The latency and energy of a tensor algebra accelerator depend on how data movement and operations are scheduled in the accelerator architecture (i.e., the *mapping* [37]). Various mapping optimizations have been proposed to leverage various features of the workload to lower latency and energy, including dataflow [10, 26, 35], tiling [16, 21, 50], and fusion [8, 16, 26, 35, 49]. Often, state-of-the-art accelerators combine several of these optimizations [26, 35, 49].

This paper, like many prior works, uses the *Einsum* notation [19, 34–36] to analyze and reveal the space of optimizations for computation steps (which we will also call *Einsums*) in a workload.

Mapping optimizations often are aimed at reducing high-energy, high-latency data movement both on and off-chip. To accomplish this, we can *reuse* data across multiple computations rather than fetching it from more costly memories. Within each *Einsum*, we can capture *intra-Einsum reuse* by optimizing computation order (i.e., *dataflow*) and data tiling [10, 29, 37]. Between *Einsums*, we can capture *inter-Einsum reuse* by *fusing* to share data across multiple *Einsums* (e.g., reusing the intermediate result between a matrix multiplication and a following softmax). Many workloads, such as Transformer self-attention, contain *Einsums* with especially low intra-*Einsum* reuse opportunity, and fusion has become a critical optimization for those workloads [11, 12, 26, 35].

However, a challenge that state-of-the-art accelerators have not adequately addressed is that mapping optimizations often involve tradeoffs between these reuse types, and the optimal choice changes depending on the workload and architecture. Even minor workload changes (e.g., changing the number of input tokens of a Transformer) can result in drastically different optimal mappings. For example, our analysis in Section 8 shows that for long sequence lengths, TransFusion [49], a state-of-the-art accelerator for Transformers, often incurs high energy by fusing too many *Einsums*. The reason is that leveraging intra- and inter-*Einsum* data reuse both require reserving on-chip buffer capacity. With shorter sequence lengths, there is less intra-*Einsum* reuse opportunity (e.g., each weight is reused for fewer tokens), and intermediate data is smaller since it scales with the number of tokens. These factors mean allocating more on-chip buffer capacity to make fusion beneficial. However, the opposite is true with longer sequence lengths.

A solution to this challenge is to use a *mapper* [37], which searches for an optimal mapping from a space of mapping choices (i.e., a *mapspace* [37]) for a given architecture and workload, but exploring mappings with fusion is particularly challenging because it requires co-optimizing the mappings of multiple *Einsums*. For a mapper to find more efficient mappings than current solutions, a *comprehensive* mapspace must be considered (i.e., various dataflow, tiling, and fusion choices). However, in such a comprehensive mapspace, there are many dataflow and tiling choices for each *Einsum*, and the number of overall mappings is the product

*These authors contributed equally to this work.

Mapper	Comprehensive mapspace	Optimal	Time to optimal
Timeloop[37] Maestro[29] Zigzag[32]	No: No Fusion	No	N/A
Optimus[8] ConvFusion[42] FusedCNN[1] DeFiNES [31] FLAT [26]	No: Limited dataflows	No	N/A
SET [7] Tileflow [50]	Yes	Near*	Slow*
This work	Yes	Yes	Fast**

*Within 1 – 10% of optimal after approx. 15,000 CPU hours (Section 7).

**Optimal after approx. 1.5 CPU hours (Section 7).

Table 1: Only this work is fast and optimal.

of the number of choices for each Einsum. This makes the number of mappings to consider increase exponentially with the number of Einsums in a workload.

Current mappers [7, 50] inadequately address this large mapspace size using black-box search algorithms (e.g., genetic algorithms [50] or simulated annealing [7]), which can be slow or even fail to converge to an optimal mapping (see row 3 of Table. 1). In Section 7, we show that these works require $\geq 15,000$ CPU hours (≈ 2 CPU years) to converge to within 1% of optimal energy-delay-product for a reasonably-sized workload (a transformer layer with 10 Einsums). Meanwhile, other mappers do not support a comprehensive mapspace, often supporting only a particular dataflow, tiling, or fusion (see rows 1 and 2 of Table 1) [1, 8, 12, 26, 29, 31, 32, 37, 42].

In this paper, we introduce the Fast and Fusiest Mapper (FFM), a fast mapper that can find optimal¹ fused mappings in a comprehensive mapspace for a user-configured architecture and workload. FFM builds a mapping by starting from partial mappings (i.e., *pmappings*) that each map only one Einsum. FFM then iteratively joins pmappings until a mapping for the full workload is created. At each step, we group and prune pmappings to keep the search space small and let us quickly find an optimal mapping.

To provide intuition about this process, Fig. 1 shows a simplified analogy with a sequence of different colored puzzle pieces (Einsums) to join together and two groupings for each color piece based on their tabs and sockets. We are constructing a jigsaw puzzle (a full mapping) from a collection of puzzle pieces (pmappings). The full puzzle (full mapping) must have exactly one piece of each color (one pmapping for each Einsum). Pieces (pmappings) can only be joined if tabs and sockets match (pmappings are *compatible* if the ways they exchange shared data match). Our objective is to minimize the sum of the numbers on the pieces (some measure of cost, e.g., energy of a pmapping). Fig. 1 (b) shows that the best combination of puzzle pieces is “red 3” and “blue 1.” We can find this solution by checking all $3 \times 3 = 9$ combinations. But as we increase the

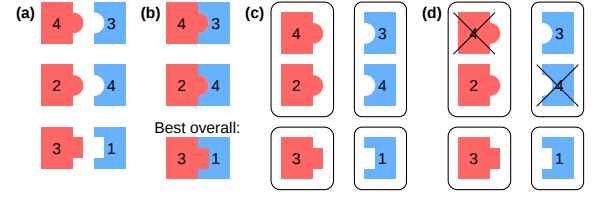


Figure 1: An analogy of pmappings as puzzle pieces. (a) Tabs and sockets represent compatibility. Numbers represent a cost metric. (b) Pieces can be joined in several ways. (c) Pieces can be grouped by tabs/sockets and (d) pruned per-group.

number of colors of pieces in the puzzle (number of Einsums in the workload), the runtime of this brute-force approach would grow exponentially, and this approach becomes infeasible.

Now, we apply two optimizations to reduce the effort to find the optimal solution. **First, we can avoid trying many incompatible pairs by grouping pieces with the same tab and socket shapes and only considering combinations within compatible groups.** As Fig. 1(c) shows, this optimization reduces the possible combinations from 9 to $2 \times 2 + 1 \times 1 = 5$. **Second, note that we only have to consider the best piece, with the lowest number, in each group.** Fig. 1(d) shows that we can maintain overall optimality while greedily selecting puzzle pieces with the lowest numbers within each group because they are the same shape. This *pruning* of pieces dramatically reduces the number of combinations to try; in the example, pruning away two pieces reduces the possible combinations from 5 to 2.

While the puzzle analogy is simple and intuitive, we need more considerations to apply to a mapspace search. We discuss these in-depth in the paper, but one example is that pruning pmappings involves a multi-faceted tradeoff. For example, pmappings often trade off resource usage (e.g., on-chip buffer usage). Thus, the lowest energy pmapping for an Einsum may not lead to the lowest energy mapping overall if that pmapping reserves too much on-chip buffer, which limits how much data pmappings of other Einsums can buffer. **To ensure we do not prune a potentially efficient pmapping, we generate all pmappings in the Pareto-frontier in terms of criteria that include objectives (e.g., energy and latency) and resource usage (e.g., memory usage).**

In summary, this paper makes the following contributions:

- We enable grouping and joining pmappings by defining the *compatibility criteria*, such that two pmappings may be joined (i.e., satisfy data dependencies) if and only if they agree in terms of compatibility.
- We enable pruning within each pmapping group by capturing optimization objectives (e.g., energy and latency) and hardware resource usage (e.g., memory usage) as *objective* and *reservation criteria* respectively. Optimality of the final, overall mapping is ensured if we only prune pmappings worse in *all* criteria (e.g., high energy and latency while using a lot of memory). Section 6.4 provides a proof.
- We introduce the Fast and Fusiest Mapper (FFM), a fast algorithm to find optimal fused mappings in a large mapspace for a wide variety of workloads. FFM constructs mappings

¹While the optimal definition is mapspace-dependent and we cannot predict future mapping optimizations, our mapspace is a superset of the mapspaces in many prior, and state-of-the-art works [1, 7, 8, 26, 29, 31, 32, 37, 42, 50].

one Einsum at a time, grouping and pruning before joining pmappings to keep the search space small.

- We show that FFM can find optimal mappings of workloads with many Einsums (experimentally verified up to 64 Einsums) despite the exponentially increasing mapspace size, demonstrating the effectiveness of our pruning.
- We demonstrate the value of FFM by showing an accelerator that uses FFM’s mappings to improve accelerator EDP by up to 1.8× compared to TransFusion [49], a state-of-the-art Transformer accelerator with hand-optimized fusion.
- We evaluate FFM against state-of-the-art automated mappers. FFM’s mappings enable 2× lower accelerator EDP compared to prior mappers when given similar search time. Furthermore, prior works do not find optimal mappings even with 10,000× more runtime than FFM.

2 Background

We briefly discuss the Einstein summation (Einsum) notation [13, 19, 34, 36], which precisely describes a wide range of tensor algebra computation steps, and the LoopTree notation [14, 16], which precisely describes a wide variety of fused (and unfused) multi-Einsum mappings.

2.1 The Einsum Notation

Computation steps in a tensor algebra workload (e.g., a matrix multiplication or a convolution step in a DNN layer) can be described using the *extended Einsum notation* [13, 19, 34, 36]. In the Einsum notation, input and output multi-dimensional data are *tensors*, and the dimensions of the tensors are called *ranks* [41]. For example, in a matrix multiplication $Z = A \times B$, the matrices A , B , and Z are the tensors, the dimensions M , K , N are the rank names and the Einsum is $Z_{m,n} = A_{m,k} \times B_{k,n}$. The variables m , n , k , which index into ranks, are called *rank variables*, and a summation is implied over rank variables that are present in the right-hand side of the equation but not in the left-hand side (k in this example). By convention, rank variables are often the lowercase letters of the rank names.

2.2 The LoopTree Notation

We explain the *LoopTree* notation [14, 16] using an example workload with two matrix-vector multiplications:

$$\text{(Step 1) Einsum A: } A_{nA} = I_{nI} \times W A_{nI,nA} \quad (1)$$

$$\text{(Step 2) Einsum B: } B_{nB} = A_{nA} \times W B_{nA,nB} \quad (2)$$

Fig. 2(a) shows pseudocode for a fused mapping for these Einsums, and Fig. 2(b) is the equivalent LoopTree mapping.

A LoopTree mapping consists of the following nodes:

- **Loop nodes** (rectangles) contain for-loops that iterate over rank variables in the workload. A loop may be shared by multiple fused Einsums (i.e., an *inter-Einsum* loop). For example, see the nA loop, which represents co-iteration of multiple Einsum computations.
- **Storage nodes** (cylinders) represent tensor tiles in memory. A storage node is annotated with the memory level (e.g., DRAM or GLB) and the tensor stored in that memory level. Loops above the storage node determine the tile shape and

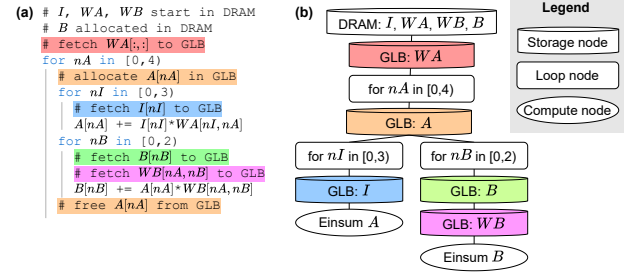


Figure 2: (a) Pseudocode of a mapping for two matrix-vector multiplications and (b) the equivalent LoopTree mapping.

iteration order. In our example, the placement of storage node “GLB: I ” indicates that the nA rank is divided into four and the nI rank into three to create twelve tiles of I . At each loop iteration, one such tile is fetched into GLB.

- **Compute nodes** (ovals) indicate processing of one computation in the Einsum (e.g., multiply-accumulate operations in this example).

In a LoopTree, important characteristics include the relative placement of nodes (e.g., **storage node over a loop node means tensor is reused across loop iterations**) and splits (e.g., **loop above split means all Einsums below split interleave execution, storage nodes above splits means all Einsums under the split can use the same stored tensor tile**).

3 From Puzzle Analogy to Mapspace Search

While the analogy in Section 1 is intuitive, it oversimplifies the problem. In this section, we will (1) describe how FFM constructs mappings from compatible pmappings, and (2) how to prune given the more complex tradeoffs of pmappings. Moreover, we will describe the *criteria* with which we evaluate pmappings to ensure compatibility in (1) and optimality in (2).

3.1 FFM Constructs Mappings by Joining Compatible Pmappings

Takeaway: FFM constructs a full mapping by iteratively joining single-Einsum mappings, called *partial mappings* or *pmappings*. The *compatibility criteria* capture whether pmappings are compatible and can be joined.

FFM constructs full mappings by iteratively joining compatible producer/consumer pmappings. In the following, we will describe the requirements for pmappings to be compatible and how pmappings are joined together. As an example, suppose we have as workload a cascade of three vector-matrix multiplications (insert after Eq. 2, a new Einsum C , $C_{nC} = B_{nB} \times W C_{nB,nC}$). Fig. 3 shows a two-step process for joining three pmappings, one pmapping for each Einsum. We will walk through this example.

First, data dependencies between Einsums constrain which pmappings are compatible. These data dependencies occur when Einsums share tensors (e.g., when an Einsum produces an intermediate that is consumed by another). For two pmappings to be compatible, they must agree on (1) the tile shape of shared tensors, (2) the memory

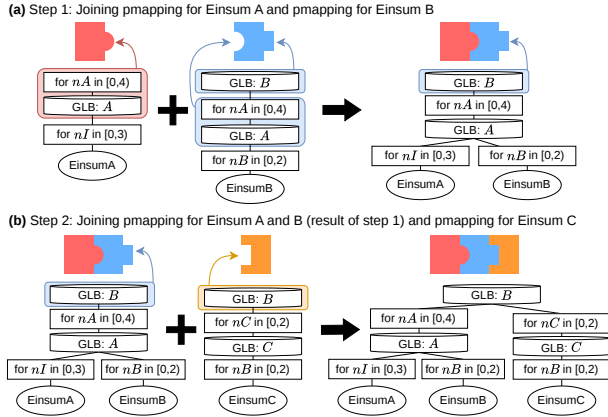


Figure 3: An example of creating a mapping in two joining steps. Weight tensors are omitted for clarity since they do not affect compatibility. (a) Joining pmappings for Einsums A and B. (b) Joining the pmapping for Einsums A, B and the pmapping for Einsum C. Storage nodes of shared tensors and the loops above them must match for pmappings to be compatible; these parts of the pmappings are highlighted in color and connected to the tabs/sockets analogy.

level that keeps those tiles, and (3) the iteration order of those tiles (*i.e.*, the dataflow). In a LoopTree, this means two mappings are compatible if they match in (only!) the storage node of shared tensors and the loops above these nodes. For example, in Fig. 3(a), a pmapping for Einsum A is joined with a compatible pmapping for Einsum B. The shared tensor for Einsums A and B is only tensor A, and these pmappings are compatible because they have the same loop over nA and storage node “GLB: A” (see the highlighted portions of pmappings and their puzzle equivalents). Note that the “GLB: B” is irrelevant to compatibility because it is not a shared tensor between Einsums A and B.

In FFM, we capture all these characteristics that are relevant to compatibility as *compatibility criteria*, which we will discuss in greater detail and precision in Section 4.

When we join pmappings, we merge their compatible portions, making it one shared portion, and place a LoopTree split underneath that shared portion (see the result pmapping Fig. 3(a)). Other portions of the pmapping (*e.g.*, the “GLB: B” node) are inserted in their corresponding places in the result². We can keep joining pmappings until we have the full mapping for the entire workload. For example, Fig. 3(b) shows the result from Fig. 3(a) being joined with a third pmapping for Einsum C. The shared tensor is now B, and the process from before can be applied in this step.

Finally, this process can construct a comprehensive mapspace: We can decompose any LoopTree mapping into individual pmappings, and conversely, there exist pmappings that join to create any given LoopTree mapping. Furthermore, there is a one-to-one correspondence between a LoopTree and a loop nest, so this procedure can be used to realize arbitrary loop nest mappings.

²There may be multiple equivalent ways to place these nodes. In a LoopTree, only the order of loop nodes and the relative positioning of storage nodes between the loop nodes matter. These are captured in the compatibility criteria definition.

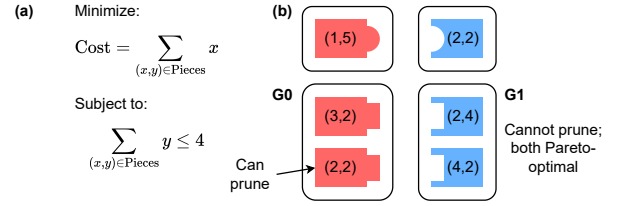


Figure 4: An example constrained optimization and (b) example puzzle pieces with two criteria (objective, reservation) to consider. Two groups of puzzle pieces, group G0 and group G1, are highlighted for discussion.

3.2 Optimal Pruning under Complex Tradeoffs

Takeaway: Pruning *within* a group maintains optimality because the criteria used to prune within a group capture all tradeoffs between pmappings. Specifically, *objective criteria* capture the contribution of a pmapping towards the overall objectives (*e.g.*, latency and energy), and *reservation criteria* capture how much hardware resources (*e.g.*, memory capacity) are reserved by the pmapping.

Unlike the simple analogy in Section 1, in which pieces can be assigned a single numerical cost, choosing pmappings involves significantly more complex tradeoffs because pmappings compete for limited hardware resources. For example, one pmapping may have low energy but reserves most of the on-chip memory. Choosing this pmapping forces pmappings of other Einsums to use what little on-chip memory remains, which may reduce data reuse and increase energy overall. In general, choosing pmappings requires consideration of the efficiency of the pmapping in terms of our overall objective (*e.g.*, latency and energy) and the amount of hardware resources reserved. For each pmapping, we capture the former using *objective criteria* and the latter using *reservation criteria*.

To provide intuition, Fig. 4 shows a puzzle analogy in which each puzzle piece has an objective and a reservation of a resource (*e.g.*, memory capacity), and the goal is finding a combination that minimizes the sum of objectives while keeping the sum of reservations less than or equal to 4. This creates a new tradeoff where a piece may have a lower objective, but reserves too much of a resource.

We can prune while guaranteeing optimality of the final solution by pruning only pieces that are worse than another piece within that in both objective and reservation criteria (proof in Section 6.4). To enable discussion, we will refer to a piece by its criteria tuple, (objective, reservation). Fig. 4(b) shows an example in which group G0 contains two pieces with criteria (2,3) and (3,3). In G0, we can prune the piece with (3,3) because (2,3) is always better, with a lower objective and the same reservation. On the other hand, we cannot prune in group G1 because one has a lower objective while the other has a lower reservation. When there is a piece that is worse than another piece in all criteria, we refer to the former piece as being *Pareto dominated*. After pruning all Pareto-dominated pieces, the remaining pieces in each group form the *Pareto frontier*.

3.3 Summary of Compatibility, Objective, and Reservation Criteria

In summary, FFM ensures joined pmappings respect data dependencies using the compatibility criteria, and evaluates the efficiency of pmappings using the objective and reservation criteria. Table 2 organizes these criteria, points to sections that contain more detail, and shows examples and how the criteria are used in FFM.

4 Compatibility Criteria

A valid mapping must satisfy data dependencies (e.g., data must be read after it is written and before it is freed). In this section, we discuss compatibility, which encompasses when and where pmappings exchange data. Moreover, we show how we encode compatibility as criteria (based on backing storage and loops above backing storage) to enable grouping and pruning within groups.

4.1 Formalizing Compatibility Criteria

Takeaway: Two pmappings are *compatible* iff they agree on every shared tensor’s (1) tile shape, (2) backing memory level, and (3) tile production/consumption order (dataflow). FFM’s representation of compatibility criteria captures a comprehensive mapspace.

Pairs of pmappings must respect data dependencies to be compatible. Specifically, **for every shared tensor, pmappings must agree on: (1) the shape of shared tensor tiles; (2) the memory level in which the tiles are stored, and (3) the order of production and consumption of the tiles (i.e., the dataflow).** These three attributes comprise the tensor’s *compatibility*.

Now, we formalize the compatibility criteria of a pmapping as the set c_T for every shared tensor T , where c_T is the 3-tuple

$$c_T = (\text{tile shape, backing storage, tile exchange order}) \quad (3)$$

In the LoopTree, **the tile exchange order (dataflow) is the order of loop nodes above the storage node, the tile shapes are the bounds of those same loops, and the backing memory (where tiles are exchanged between pmappings) is the highest storage node (outer-most memory level) that stores tensor T .**

Importantly, this representation encompasses a comprehensive mapspace. For example, FFM allows any loop order as dataflow, which is much more flexible than many state-of-the-art designs, which only allow certain orders or impose a fixed dataflow [11, 12, 17, 26, 31, 35]. Moreover, the backing memory choice encompasses the fusion design space. A pmapping can specify a backing memory to be DRAM (making it unfused), GLB (making it fused), or any other memory level. This lets us support a comprehensive fused space, including fusion at every memory level, rather than treating fusion as a separate case that must be considered. This choice is also more flexible than many state-of-the-art designs [11, 12, 26, 35, 49].

4.2 Compatibility for Grouping and Skipping Incompatible Joins

Takeaway: Grouping based on compatibility lets us skip many incompatible joins and ensure pruning optimality because all pmappings in a group impose identical compatibility constraints on future pmappings.

After generating the compatibility criteria for all pmappings, **we group pmappings with the same compatibility** to enable two optimizations: skipping joining incompatible pmappings and optimality-preserving pmapping pruning.

We reduce the time spent checking for pmapping compatibility by only joining pmappings from compatible groups. For example, instead of trying to join every pmapping for Einsum A with every pmapping for Einsum B , we check the compatibility between groups for Einsum A and groups for Einsum B . Then for any compatible group of A and group of B , any pmapping in the group of A is always compatible with any pmapping in the group of B .

Moreover, **grouping based on compatibility ensures that pruning is optimality-preserving because any two pmappings in the group are interchangeable.** Specifically, just as we can trade one puzzle piece for another in the same group in Fig. 4, we can trade any pmapping in a mapping with another pmapping from the same group. So, if pmapping p_1 is Pareto-better than pmapping p_2 in the same group, it is optimality-preserving to prune p_2 because any mapping containing p_2 could be improved by replacing p_2 with p_1 .

5 Reservation Criteria

Takeaway: Processing an Einsum requires reserving hardware resources. While joining pmappings, we track the current reservations at every possible lifetime, using them as pruning criteria.

Valid mappings must reserve system resources (e.g., buffer capacity, number of processing elements) within available limits. Reservation limits represent a constraint, but also a pruning opportunity. All other attributes being equal, a pmapping that reserves more resources is a worse pmapping and a pruning candidate.

In this section, we discuss (1) calculating resource reservations from a *full* mapping, (2) tracking the minimum amount of reservation information in a *partial* mapping to still be able to compute the final reservations, and (3) pruning *partial* mappings using reservation information.

5.1 Calculating Reservations of Full Mappings

Takeaway: Overall reservations can be computed by summing or taking the maximum of the reservations of each pmapping.

To illustrate how we calculate a mapping’s reservations, we will calculate GLB usage for the mapping in Fig. 5(a). We first convert the LoopTree to *ReservationTree*; storage nodes immediately above splits are replaced with *reservation nodes* in branches where the reservation is live (e.g., “GLB : A ” in Fig. 5(a)). Other storage nodes are replaced with reservation nodes at the same spot (e.g., “GLB : WA ” in Fig. 5(a)).

Fig. 5(b) shows the resources reserved by the ReservationTree. Memory usage is not uniform during the processing because the sizes and lifetimes are different for different tensor tiles, but the maximum memory usage (dotted line) must be within the GLB capacity for the mapping to be valid.

All reservations within a branch are live at the same time, so we can calculate GLB usage for a branch as the sum of reservations in the branch (e.g., Fig. 5(a) has GLB reservations for A and I , and Fig. 5(b) shows A and I stacked in the same timestep). Different

Criteria (Section)	Example	Puzzle Analogy	FFM Use	Overall Goal
Objective (Section 3.3)	Pmapping Energy	Piece cost	Prune in group	Minimize
Compatibility (Section 4)	Shape of shared tensor tile and which memory level the exchange occurs and iteration order of tiles	Tab/socket shape	Group	Satisfy data dependencies
Reservation (Section 5)	Tensor X memory usage and for how long Tensor X is alive	Piece weight	Prune in group	Ensure below resource limits

Table 2: A taxonomy of criteria that we use to compare and prune pmappings. Examples are illustrative, not full descriptions.

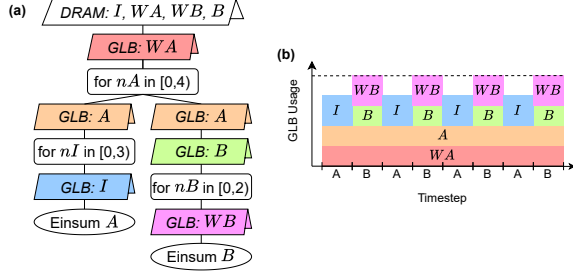


Figure 5: (a) ReservationTree for the mapping in Fig. 2(b) replaces storage nodes (cylinders) with reservation nodes (placards), which show tensor tile lifetimes across branches explicitly. (b) Illustration of GLB usage. Color and label correspond to reservation nodes. Timesteps are marked with the Einsum being processed. The dashed line shows the maximum GLB usage.

branches are active at different times, so we take a maximum of the GLB usage across branches (e.g., in Fig. 5(b), compare GLB usage for Einsum A and B). This pattern generalizes: To calculate the total reservations of a ReservationTree, recursively sum reservations within a branch, and max reservations across branches.

5.2 Reservation Lifetimes and Consolidating Reservations

Takeaway: Each Einsum adds additional reservations. We can consolidate reservations with the same lifetime, letting us track only a limited number of reservations and avoid slowing down the mapper with too many pruning criteria.

The challenge in our method is that a pmapping alone does not tell us the lifetime of all reservations in the final mapping. Fig. 6(a) shows the same ReservationTree as in Fig. 5, but now as a pmapping, and there are two places where a branch may be joined in the future: BranchC and BranchD. A branch may be added under any loop or under the outermost DRAM storage node. In this example, we know the lifetime of the tensor I tile in GLB is only within the timesteps when we process Einsum A. However, we do not yet know the lifetime of WA. If another Einsum branches off at BranchC, then WA will be live while processing that Einsum. But if that Einsum branches off at BranchD, then WA will only be live for Einsums A and B, but not the newly joined Einsum.

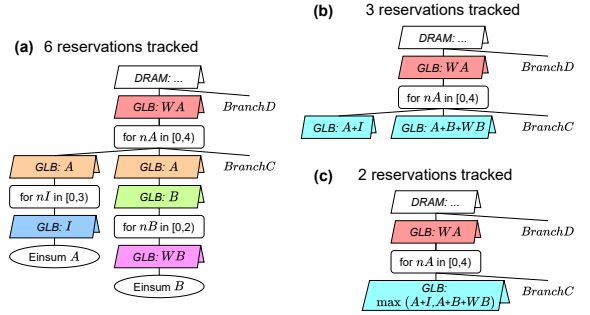


Figure 6: (a) The ReservationTree from Fig. 5 plus branches where future pmappings may be joined. (b) The ReservationTree after summing same-lifetime reservations. (c) The ReservationTree after maxing reservations across branches.

These unknown lifetimes affect GLB usage, which we address by tracking each reservation. If $\hat{C}$ is GLB usage within BranchC, $\hat{D}$ is GLB usage within branch D, and X is the memory usage of tensor X , then total memory usage is:

$$\text{Usage} = \max(\hat{D}, WA + \max(A + I, A + B + WB, \hat{C})) \quad (4)$$

Note that although we do not yet know what $\hat{C}$ and $\hat{D}$ are, tracking the reservation of each tensor separately will allow us to compute the usage at the end.

This method requires tracking many reservations, but we can reduce this number using two observations: (1) there are only a few unique lifetimes (determined by branches), and (2) many lifetimes can be consolidated. First, notice that some lifetimes include neither BranchC nor BranchD, others include BranchC only, and others include all branches. When we compare pmappings, we can only compare reservations that have the same lifetime, and two reservations have the same lifetime if they are above the same branches. This is true regardless of the contents of the branches, so it works even if we later add unknown pmappings to branches. So, any reservation between BranchC and BranchD splits could be compared to our WA reservation because they would have identical lifetimes.

We can use the relatively few unique lifetimes to reduce the number of reservations to track. Fig. 6(b) shows that instead of tracking A, I, B, and WB separately, we track the per-branch total:

$$\text{Sum same lifetimes: } \hat{A} = A + I, \hat{B} = A + B + WB \quad (5)$$

Then, we only have to track fewer values:

$$\text{Usage} = \max(\hat{D}, WA + \max(\hat{A}, \hat{B}, \hat{C})) \quad (6)$$

Moreover, branches that cannot have future pmappings joined to them (*i.e.*, branches on the left) can have their reservations consolidated into one value. Fig. 6(c) shows that the branches for Einsums A and B can be consolidated:

$$\text{Max across branches: } \widehat{AB} = \max(\hat{A}, \hat{B}) \quad (7)$$

Then, we reduce the number of values to track even more:

$$\text{Usage} = \max(\hat{D}, WA + \max(\widehat{AB}, \hat{C})) \quad (8)$$

A crucial benefit of consolidation it means **the number of reservations to track does not depend on the number of Einsums**. If the next split occurs $\leq N$ loops down, then we only need to track up to $2N + 1$ reservations.³ The split point is limited by the number of loops above the current split in each pmapping, which is limited by the number of shared ranks between the most recent and next Einsum. This number is generally small (*e.g.*, 4 for transformers).

5.3 Pruning Pmappings using Reservations

To prune using reservations, we need to consider both the size of each reservation and the time period for which each reservation is alive. First, note that reservations can be compared for two pmappings which have the same expression for resource usage (*e.g.*, the same expression as Eq. 8 just with different values for WA and AB). Equivalently, two pmappings can be compared if they have the same ReservationTree form after consolidation (*e.g.*, the tree in Fig. 6(c)). Note that a ReservationTree after consolidation reduces to a subtree that contains the same information as the compatibility criteria. **Therefore, any two pmappings with the same compatibility criteria can have their reservations compared.**

To compare the reservation of two pmappings and ensure pruning preserves optimality, we consider all reservations and check for Pareto dominance (*i.e.*, a pmapping has worse reservations if and only if its reservation is larger across all lifetimes for all resources).

6 Fast and Fusiest Mapper (FFM)

Takeaway: FFM works in two phases: (1) *single-Einsum exploration* generates Pareto-optimal pmappings for each Einsum and (2) *iterative group-prune-join* processes one Einsum at a time. Iterative pruning limits the active Pareto-frontier size, which reduces mapper runtime and memory overhead.

The Fast and Fusiest Mapper (FFM) implements the ideas in the previous sections to quickly find optimal mappings. Fig. 7 shows FFM, which has two main steps. **First, FFM explores the space of pmappings for each Einsum.** Next, **starting with the first Einsum, it repeatedly groups, prunes, and joins pmappings until all Einsums are joined.**

6.1 Fast Single-Einsum Pmapping Generation

We generate all Pareto-optimal pmappings for every Einsum, exploring dataflow, tile shape, storage node placement, and spatial

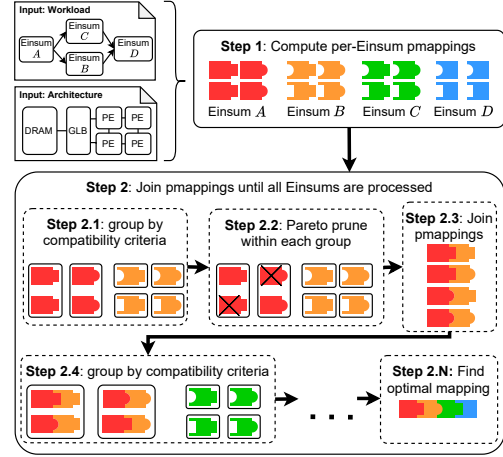


Figure 7: Overview of the Fast and Fusiest Mapper (FFM).

allocation. To explore all fusion options, the pmapping exploration considers all backing storage choices.⁴

While an exhaustive exploration of all pmappings would be slow [37], we use TCM [15], which can quickly generate Pareto-optimal pmappings by pruning the search space by more than 30 orders of magnitude.

6.2 Group, Prune, and Join Pmappings

We explain this section with an example, then extend it to the full algorithm. Assume we have the pmappings for Einsum A and we'd like to join them with the pmappings for Einsum B. We would perform the following steps:

- **Group** the pmappings for Einsum A based on compatibility criteria. Do the same for Einsum B.
- **Prune** dominated pmappings in each group by eliminating pmappings not on the Pareto frontier.
- **Join** compatible pmappings for each group combination. For every valid combination of groups, we generate every combination of pmappings.

After performing these steps, we will have pmappings that map both Einsums A and B. We repeat the process, adding pmappings for one Einsum at a time until we have processed all Einsums.

We use several optimizations to speed up this process. First, a given pmapping for one Einsum will likely only be compatible with a fraction of pmappings for another Einsum, so we **skip incompatible joins** by pairing groups of pmappings first based on their compatibility, then joining pmappings only between compatible groups (described in Section 4).

Next, each Einsum adds more reservations to track, so after each join step we **consolidate reservations** (described in Section 5) to keep pruning effectively.

³The factor 2 comes from the fact that two lifetimes must be tracked for every potential split: reservations with a lifetime that end before future Einsums and reservations with a lifetime that continues to future Einsums. The constant 1 is for reservations below the bottommost split, which never live through future Einsums.

⁴While we show one processing order for the Einsums, there may be more than one data-dependency-satisfying processing order of Einsums. FFM can explore these orders by joining in all data-dependency-respecting orders of Einsums. Pmappings are the same for any order and are only computed once.

6.3 Why Per-Einsum Runtime Does Not Increase With More Einsums

The astute reader may wonder, if the number of mapping choices increases exponentially with more Einsums, how does FFM stay fast? In practice we find that the runtime scales approximately linearly with the number of Einsums for two reasons.

First, the runtime of generating pmappings is linear in the number of Einsums because it is performed independently for each Einsum. This runtime generally dominates the overall FFM runtime.

Joining runtime does not increase because we keep the number of pmappings bounded in each joining step. To do so, we note that we keep pmappings that are Pareto-optimal in terms of compatibility, reservations, and objective metrics. There are finite compatibility choices (because they depend on the loops of only the most recent Einsum) and finite reservations (because we perform consolidation), so the number of dimensions in the Pareto space is bounded. We only need to ensure that the density of the Pareto-optimal space remains bounded.

To keep the Pareto-optimal space from becoming arbitrarily dense, we use ϵ -pruning [30] to perform a “dirty” join, yielding mappings within $(1+\epsilon)\times$ optimal in the chosen objective. ϵ -pruning coarsens the Pareto frontier to bound the number of points [30]. However, it may yield mappings up to $(1+\epsilon)\times$ optimal in each objective, so we use two strategies to preserve optimality:

- *Resource Reservations:* If ϵ -pruning reservations yields a valid mapping, then it is known to be optimal because we only care if reservations are valid (e.g., if a mapping has the lowest energy and has valid resource usage, we don’t care whether the resource usage could be ϵ lower). If ϵ -pruning yields an invalid mapping, we retry with a smaller ϵ .
- *Objectives:* We first perform a dirty search with ϵ -pruning, yielding a near-optimal mapping with objectives $\leq (1+\epsilon)\times$ optimal. Then we perform a second clean search and, after each joining step, prune pmappings worse than our near-optimal mapping. Most pmappings are significantly worse than optimal [37], so this pruning removes most pmappings.

In cases where there is one objective metric (e.g., energy or latency) and dirty resource reservation pruning succeeds for $\epsilon > 0$, the number of mappings stays bounded and we can guarantee runtime is linear with number of Einsums.

In theory, runtime may be exponential with the number of Einsums if dirty resource reservation pruning fails or if we have multiple objective metrics (e.g., optimize both energy and latency $\rightarrow$ runtime may increase with the number of mappings on the energy-latency Pareto). In practice, however, we have not seen cases that scale superlinearly. Dirty resource reservation pruning nearly always succeeds on our first ϵ guess of 0.2, and we have never seen it fail for $\epsilon \leq 0.02$. Meanwhile, multi-objective optimization has been fast for all tested workloads (e.g., all experiments in this paper use FFM returning the full energy-latency-optimal Pareto, and Fig. 9 shows that runtime does not increase).

6.4 FFM finds Optimal Mappings

FFM finds optimal mappings in our mapspace because it (1) explores every non-pruned mapping and (2) ensures all pruning is optimality-preserving. Here, we prove both of these statements.

To satisfy (1), we explore every pmapping for every Einsum with TCM [15], which can fully explore mapspaces. Then, when joining, we enumerate every valid combination of pmappings, thus yielding every possible mapping choice.

To prove (2), we show that if pmapping X is pruned, then for any mapping containing X , there exists another mapping that is both (A) valid, data-dependency-satisfying, and same-or-better in every objective metric, and (B) not pruned.

Proof of (A): X is pruned if and only if there exists a pmapping Y that dominates X , meaning X and Y have identical compatibility and $Y \leq X$ in every resource usage and metric. Therefore, any valid mapping that contains X could use Y instead, and the result would be valid ($Y \leq X$ in usage of every resource), satisfy data dependencies (Y and X have identical compatibility), and be better ($Y \leq X$ in objective metrics).

Proof of (B): Define the set S of pmappings that dominate X . If $S_0 = \emptyset$, then X is non-dominated and is not pruned. Otherwise, prove by induction. Assume true for $|S| < n$. For $|S| = n$, pick $Y \in S$ and define S' as the set of pmappings that dominate Y . Any pmapping that dominates Y also dominates X , and Y does not dominate itself, so $|S'| < |S| = n$. By our assumption, either Y is non-dominated or S' contains a non-dominated pmapping.

7 Comparison to Prior Mappers

Takeaway: FFM quickly finds optimal mappings, while prior mappers fail to do so, even in orders-of-magnitude more runtime. This is because an optimal mapping must balance many tradeoffs across memory levels and Einsums.

7.1 Accelerator Architecture & Validation

We evaluate using an architecture based on TPUv4i [24], which has a 128 MiB global buffer and four cores. Each core has a 4 MiB local buffer and a 128×128 PE-array that supports 8-bit multiply-accumulate (MAC) operations, operating with a weight-stationary dataflow. The architecture operates at 1.05 GHz, and connects to DRAM with 614 GB/s bandwidth.

Hardware Model: The hardware model is based on the validated LoopTree [16] model and component energies are modeled with HWComponents [4, 48]. The energy and throughput of all components are taken directly from the TPUv4i [24] paper. We validate our model against LoopTree, Timeloop, and CiMLoop [4, 16, 37], and it yields identical results. Additionally, FFM’s open-source release includes the full CiMLoop suite, with models of five fabricated [23, 27, 40, 43–46] and four simulated [3, 38, 39, 51] chips, validated to match energy, area, and latency within 10% of published data under a range of workloads and operating conditions (e.g., varied supply voltage).

7.2 Baselines

We compare our mapper to three baselines implemented based on prior mappers⁵.

SET [7], uses simulated annealing to explore storage node placements, shared loops for each Einsum, and intra-Einsum by randomly changing choices and accepting the new one probabilistically. Cooling rate and initial temperature are taken from the SET implementation [47].

TileFlow [50] uses a genetic algorithm to pick storage node placements, shared loops for each Einsum, and intra-Einsum mapping choices. Each population has 10 randomly-initialized mappings (2000 total across 192 threads)⁶. We set a crossover rate of 0.7 and a mutation rate of 0.2, which are the best-converging options found for eight matrix multiplications.

Timeloop [37] randomly samples a pmapping for each Einsum.

7.3 Methodology

For **FFM** and all baselines, we report the energy-delay-product (EDP) of the mappings found and the runtime of each mapper. Runtime reported for **FFM** is the wall-clock time, which includes the exploration of single-Einsum pmappings and the process of joining and pruning pmappings.

Baselines would take CPU-years to run directly, so we generously estimate runtime (a lower bound of actual runtime). To estimate baseline runtime, we run the baselines, but replace their mapping evaluation calls with queries to the cached results from **FFM**'s pmapping explorations. We model baseline runtime by multiplying the number of queried pmappings by the amount of time it takes to evaluate a pmapping.

This estimated runtime is a generous underestimate for multiple reasons. First, we ignore the time taken by the baselines to join pmappings and to select new pmappings (e.g., for **TileFlow**, we do not include the time required to perform crossover and mutation). Second, we let the baselines benefit from **FFM**'s batched pmapping generator calls (asking the pmapping generator for all Pareto-optimal pmappings, rather than one pmapping at a time), which speeds up the pmapping generator by orders of magnitude [15]. Third, we only give baselines compatibility-valid pmappings, dramatically reducing their required search space. Baselines only initialize mappings with compatible choices, and when **TileFlow** or **SET** transform one pmapping, other pmapping(s) are also transformed into compatible equivalents.

Experiments ran on 2 Intel Xeon Gold 6252 and 384GB of RAM.

7.4 FFM Quickly Finds Optimal. Baselines Cannot Even With 10,000× More Time

We compare to the baselines mapping to TPU-v4i [24] the GPT-3 6.7B Transformer model [6] with batch size 64 and 4096 tokens. The batch size and token count represent a common scenario in which shared tensors do not fit in on-chip memory without tiling them, making it nontrivial to find a good fused mapping.⁷ **FFM**

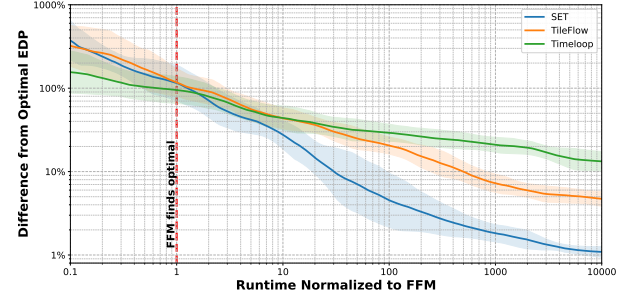


Figure 8: Convergence speeds of baselines relative to FFM. FFM finds the optimal mapping while baselines are still 100% away from optimal EDP. Baselines are stochastic, so we show the average and ± 1 standard deviation range of ten runs. Baselines are $\geq 1\%$ away from optimal EDP even at 10,000× FFM’s runtime. FFM ran in ≈ 1.5 CPU hours.

finished after 1.5 CPU hours. We terminate baselines after 10,000× the time taken by **FFM**.

Fig. 8 compares how fast the baselines converge to the optimal EDP relative to **FFM**. The y-axis shows the percent difference from optimal EDP. We can see that by the time **FFM** completes, the baselines are approximately 100% away from optimal (2× optimal). Given additional time, baselines converge slowly. **SET**, the best-performing mapper, is 30% away from optimal after 10× our runtime and 6% off optimal after 100× our runtime. To yield a mapping within 1% of optimal, **SET** requires 15,000 CPU hours (1.7 CPU years), making search infeasible for most design space explorations.

7.5 FFM is Fast Even with Many Einsums

To compare the speed of our mapper against the baselines as a function of the number of Einsums, we run the mappers on chains of matrix multiplications (the output of one matrix multiplication is the input to the next and so on), and vary the number of matrix multiplications. Each matrix multiplication has $M = 8,192$; and we create a variety of tensor shapes by using the following pattern for $(N; K)$: $(16,384; 16,384) \rightarrow (4,096; 16,384) \rightarrow (4,096; 4,096) \rightarrow (16,384; 4,096) \rightarrow \text{repeat}$.

We time out baselines after 1000× **FFM**'s runtime. Like in the previous experiment, we average the results of ten runs for the baselines. When five or fewer of the ten runs time out, we generously assume the timed-out runs finish in **FFM**'s runtime. When six or more time out, we consider the baseline to have timed out.

Fig. 9 shows the time taken by **FFM** to find an optimal mapping and the baselines to find a mapping within 5% of optimal. **FFM** quickly finds optimal mappings for all numbers of Einsums, while the runtime of the baselines rapidly increases with more Einsums.

In fact, **FFM**'s per-Einsum runtime is approximately constant, meaning that runtime scales linearly with more Einsums, despite the exponential mapspace size. Baseline runtime increases exponentially with more Einsums and time out. Here, we used a near-optimal threshold of 5% to get more baseline data, since thresholds closer to optimal led to baselines timing out in as few as two Einsums.

Theoretical guarantees of linear time are nuanced, so we refer the reader to Section 6.3 for a deeper discussion.

⁵Each paper’s implementation supports a different mapspace, so we create mappers for this mapspace based on their approach.

⁶We evaluated larger population sizes, but those converged more slowly.

⁷If shared tensors fit on-chip, one may fuse by keeping them on-chip without tiling.

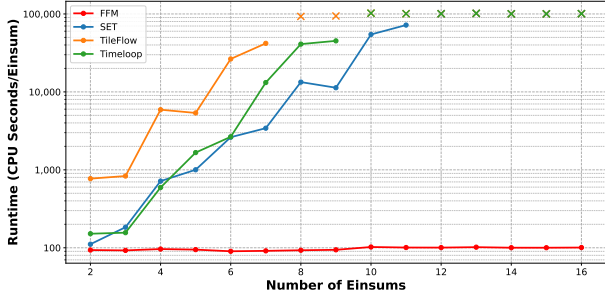


Figure 9: FFM is fast and per-Einsum runtime remains flat even with more Einsums. The plot shows runtime for FFM to find optimal mappings and baselines to find a mapping within 5% of optimal. $\times$ denotes baseline timeouts ($> 1000\times$ FFM runtime). Baseline runtime grows exponentially with more Einsums.

7.6 FFM Finds Efficient Tradeoffs Across Einsums and Memory Levels

Because of how we group, join, and prune, FFM produces mappings that achieve efficient tradeoffs across Einsums in the workload and memory levels in the architecture. In this section, we highlight some of these tradeoffs. For this discussion, Fig. 10(a) shows a subset of GPT-3 Einsums that we discuss here. Moreover, Fig. 10(b) shows the memory levels of the TPUV4i accelerator. Finally, Fig. 10(c) shows the FFM mapping with most of its nodes omitted to highlight the ones we will discuss. We make two observations.

First, we show how FFM leverages per-Einsum reuse opportunities to efficiently utilize resources shared across Einsums. Nodes (I) in Fig. 10(c) exemplify this fact. Notice that tensors from various Einsums are kept in the on-chip GLB. However, there is not enough GLB capacity to keep the entirety of all these tensors. Instead, as shown, some tensors are tiled, and different tensors are tiled differently in a way that leverages particular reuse opportunities. For example, because different elements in a batch reuse weights, weights are kept in entirety in the GLB to be reused across a batch. In contrast, tensors such as I , Q , and K (which do not have reuse opportunity across batch) are tiled, and only one element in a batch is in GLB at a time. Similarly, tensors QK and $QK_{softmax}$ are tiled in the heads rank since there is no reuse across heads of these tensors.

Second, FFM finds efficient tradeoffs across the memory hierarchy. Nodes (II) and (III) in Fig. 10(c) show loops over $m1$ and $m0$, which iterate over different tokens. The loop “S-for $m1$ ” indices parallelization over LLBs in different cores. The loop “for $m0$ ” indicates iterations over tokens at the PE level, which reuses the weights kept in PE register Reg. This full utilization of LLBs (all 4 are used) and efficient reuse in Reg ($1024\times$ reuse of WQ) is possible because the tiles of I and Q have enough tokens. If a smaller tile is kept on-chip in the GLB (a decision made by nodes (I)), there would not have been as many tokens to parallelize or to reuse weights with.

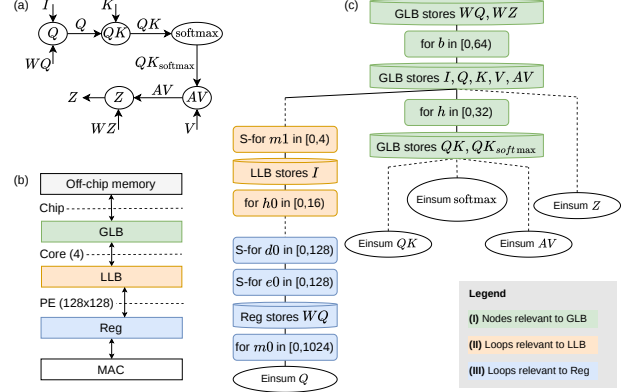


Figure 10: FFM balances tradeoffs across Einsums and across the memory hierarchy. (a) A subset of the GPT-3 workload we will focus on. Ovals are Einsums and arrows are tensors. (b) The memory hierarchy of the TPUv4i architecture. There are four cores, and each core has a 128×128 PE array. (c) The relevant subset of the FFM mapping. Dashed lines indicate omitted nodes. Certain nodes are highlighted for discussion.

8 An Optimized Accelerator Built Using FFM

Takeaway: We propose a transformer accelerator with novel mappings, generated using FFM from a comprehensive mapspace, which achieves lower energy-delay-product (up to $1.8\times$ for GPT-3) compared to TransFusion, a state-of-the-art baseline with hand-optimized fusion.

We design an accelerator for edge inference using FFM to determine the optimal mapping, which includes deciding which Einsums to fuse. The accelerator, which we refer to as **FFM**, is organized like the TPU-v4i [24] and sized to achieve ~ 33 TOPS, comparable to current edge accelerators [33]. It consists of a 5 MB global buffer, a 128×128 matrix-multiplication unit, and a vector unit with 256 compute units, which handles softmax and normalization. The accelerator is connected to an off-chip LPDDR4 [2, 4, 48] memory. We estimate GlobalBuffer energy using the Accelergy [4, 48] interface to CACTI [5]. Tab. 3 shows energy and bandwidth details.

As baseline, we compare against **TransFusion** [49], a state-of-the-art Transformer accelerator. Importantly, **TransFusion** always fuses all Einsums except K and V . We also implement the dataflow in [49], which produces and immediately consumes intermediate tensor tiles one at a time, refetching weights, K , and V as needed (*i.e.*, an intermediate-tensor-stationary dataflow). Finally, we provision the baseline to match **FFM**’s number of compute units and on-chip buffer size. We evaluate both accelerators on GPT-3 6.7B [6] with a batch size of 1.

Fig. 11 shows that **FFM** improves EDP, energy, and latency relative to **TransFusion**. Improvements increase at longer sequence lengths (up to $1.8\times$ lower EDP). From Fig. 12(a), which breaks down energy by component, we can see that FFM reduces off-chip memory energy significantly. Since **FFM** benefits from reducing data movement, there is a greater reduction in energy than in latency. This is because reducing data movement always helps energy, but only helps latency until an Einsum becomes compute-bound.

To understand the source of **FFM**'s improvements, note that there are two types of reuse opportunities: inter-Einsum (fusion) and intra-Einsum (one Einsum reusing a tensor multiple times). **FFM** balances optimizing for both types, while **TransFusion** primarily focuses on maximizing inter-Einsum reuse by fusing all Einsums except K and V , often at the cost of intra-Einsum reuse because they compete for on-chip buffer memory.

The impact of balancing intra- and inter-Einsum reuse can be seen in the off-chip energy breakdown by tensors in Fig. 12(b). Note that **TransFusion**, which focuses on inter-Einsum reuse, always keeps on-chip shared tensors other than K and V (see "Intermediates (other)"). **TransFusion** keeps these intermediates on-chip at the cost of intra-Einsum reuse, which suffers from a lack of on-chip buffer capacity, and therefore increasing "Weights" and "Intermediates (K, V)" energy. In contrast, **FFM** fuses fewer Einsums, which increases data movement of "Intermediates (other)", but frees on-chip buffer capacity to increase intra-Einsum reuse, keeping "Weights" and "Intermediates (K,V)" energy low.

We can gain more insight into how **FFM** balances leveraging intra- and inter-Einsum reuse by analyzing which Einsums are fused in the resulting mapping. Fig. 13 shows **FFM**'s fusion choices for GPT-3 with the longest and shortest sequence lengths. We make two observations. First, Fig. 13 shows that in both sequence lengths, **FFM** prioritizes fusing Einsums with the lowest compute intensity first (i.e., low intra-Einsum reuse opportunities), which gains the most benefit from fusion. This choice contrasts with **TransFusion**, which fuses all Einsums except K and V . As shown before, **FFM**'s choice, which balances intra- and inter-Einsum reuse, results in lower energy overall.

Second, the optimal balance of intra- and inter-Einsum reuse depends on workload shape, which changes the amount of each reuse type and the costs of achieving them. For example, Fig. 13

Component	Type	Energy	Bandwidth
Off-chip memory	LPDDR4	8 pJ/b	30 GB/s
Global buffer	SRAM	0.2 pJ/b	512 GB/s
MAC	int8-MAC	0.08 pJ/b	1 GHz*

Table 3: Energy and bandwidth (*frequency for compute units) of accelerator components.

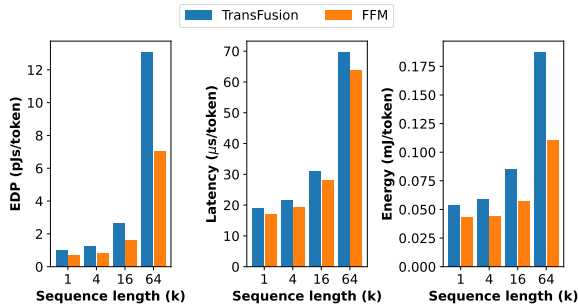


Figure 11: FFM reduces EDP, latency, and energy per token generated compared to TransFusion [49] for GPT-3.

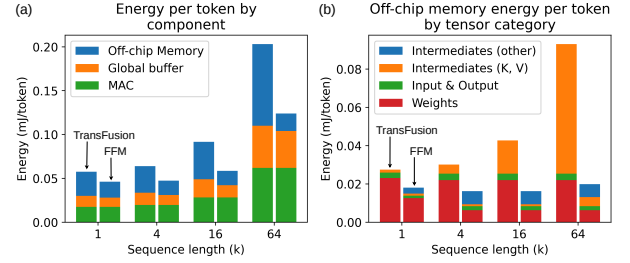


Figure 12: (a) FFM reduces off-chip memory energy compared to TransFusion. (b) FFM picks a fusion choice that balances the data movement of various tensors, for a lower overall energy. TransFusion always fuses intermediates other than K and V (notice no "Intermediates (other)").

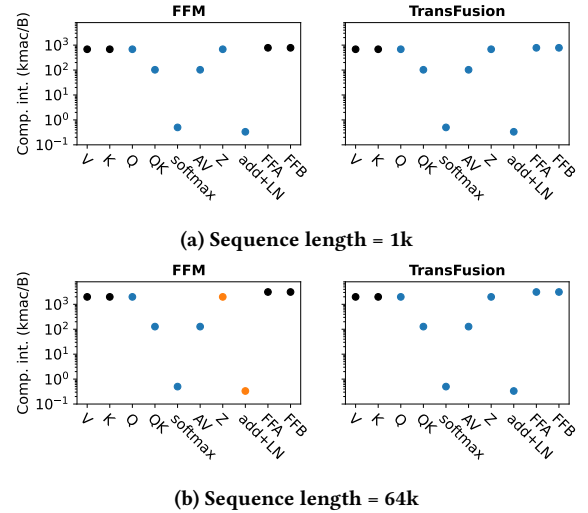


Figure 13: FFM strategically fuses Einsums with low compute intensities, which benefits most from fusion. The x-axis shows the Einsums of a GPT-3 layer and the y-axis shows the computational intensity. Unfused Einsums are in black, and different sets of fused Einsums are given different colors. Sequence length is 64k tokens.

shows that **FFM** does not fuse Einsum AV with Z when the sequence length is longer, keeping the intermediate tensor off-chip instead. This decision is made for two reasons. (1) The size of the intermediate tensor between Einsums AV and Z to keep on-chip increases with sequence length (i.e., the cost of fusion is higher). (2) Longer sequence lengths increase the amount of intra-Einsum reuse, lowering the relative benefit of fusion. Given these tradeoffs and limited on-chip capacity, **FFM** does not fuse AV and Z at longer sequence length. This is why a single choice, even an optimized one such as **TransFusion**, will not be optimal in every scenario.

9 Related Works

Many prior works propose mappers for single-Einsum workloads [20, 22, 25, 29, 32, 37]. Multi-Einsum mappers have also been proposed

for particular workloads and/or dataflows (commonly only convolutional neural networks) [1, 8, 31, 42]. Most recently, mappers that support a wider range of workloads and a larger mapspace (e.g., more dataflow choices) have been proposed [7, 50]. However, these mappers explore their large mapsaces with black-box optimization methods, which we have shown to converge infeasibly slowly.

Optimal mappings generated by FFM can be translated into loop nests, which can guide compiler tool chains [9, 18, 28].

10 Conclusion

Driven in part by the rise of deep neural networks and big data, tensor algebra workloads have become critical to accelerate, and even small differences in energy or latency can become great when these workloads are deployed at scale. Moreover, fusion has emerged as a key optimization. However, it is a challenge to find optimal fused mappings due to the exponential mapspace size. The Fast and Fusiest Mapper partitions this exploration by introducing the notion of partial mappings and defines attributes (e.g., compatibility, resource consumption, costs) that capture how they contribute to full mappings. This lets the Fast and Fusiest Mapper quickly find optimal fused mappings in a comprehensive mapspace.

References

- [1] Manoj Alwani, Han Chen, Michael Ferdman, and Peter Milder. 2016. Fused-layer CNN accelerators. In *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 1–12. <https://doi.org/10.1109/MICRO.2016.7783725>
- [2] Tanner Andrusis. [n. d.]. *HWComponents*. <https://github.com/Accelergy-Project/hwcomponents>
- [3] Tanner Andrusis, Joel S. Emer, and Vivienne Sze. 2023. RAELLA: Reforming the Arithmetic for Efficient, Low-Resolution, and Low-Loss Analog PIM: No Retraining Required!. In *Proceedings of the 50th Annual International Symposium on Computer Architecture (Orlando, FL, USA) (ISCA '23)*. Association for Computing Machinery, New York, NY, USA, Article 27, 16 pages. <https://doi.org/10.1145/3579371.3589062>
- [4] Tanner Andrusis, Joel S. Emer, and Vivienne Sze. 2024. CiMLoop: A Flexible, Accurate, and Fast Compute-In-Memory Modeling Tool. In *2024 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 10–23. <https://doi.org/10.1109/ISPASS61541.2024.00012>
- [5] Rajeev Balasubramanian, Andrew B. Kahng, Naveen Muralimanohar, Ali Shafiee, and Vaishnav Srinivas. 2017. CACTI 7: New Tools for Interconnect Exploration in Innovative Off-Chip Memories. *ACM Trans. Archit. Code Optim.* 14, 2, Article 14 (June 2017), 25 pages. <https://doi.org/10.1145/3085572>
- [6] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems*, Vol. 33. Curran Associates, Inc., 1877–1901. <https://papers.nips.cc/paper/2020/hash/1457c0d6bfc4967418bfb8ac142f64a-Abstract.html>
- [7] Jingwei Cai, Yuchen Wei, Zuocong Wu, Sen Peng, and Kaisheng Ma. 2023. Inter-Layer Scheduling Space Definition and Exploration for Tiled Accelerators. In *Proceedings of the 50th Annual International Symposium on Computer Architecture (Orlando, FL, USA) (ISCA '23)*. Association for Computing Machinery, New York, NY, USA, Article 13, 17 pages. <https://doi.org/10.1145/3579371.3589048>
- [8] Xuyi Cai, Ying Wang, and Lei Zhang. 2022. Optimus: An Operator Fusion Framework for Deep Neural Networks. *ACM Trans. Embed. Comput. Syst.* 22, 1, Article 1 (oct 2022), 26 pages. <https://doi.org/10.1145/3520142>
- [9] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. USENIX Association, Carlsbad, CA, 578–594. <https://www.usenix.org/conference/osdi18/presentation/chen>
- [10] Yu-Hsin Chen, Joel Emer, and Vivienne Sze. 2016. Eyeriss: A Spatial Architecture for Energy-Efficient Dataflow for Convolutional Neural Networks. In *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*. 367–379. <https://doi.org/10.1109/ISCA.2016.40>
- [11] Tri Dao. 2024. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=mZn2Xyh9Ec>
- [12] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *arXiv e-prints*, Article arXiv:2205.14135 (May 2022), arXiv:2205.14135 pages. <https://doi.org/10.48550/arXiv.2205.14135> arXiv:2205.14135 [cs.LG]
- [13] A. Einstein. 1916. The Foundation of the General Theory of Relativity. *Annalen der Physik* 354, 7 (1916), 769–822. <https://doi.org/10.1002/andp.19163540702>
- [14] Michael Gilbert. 2023. *LoopTree: Enabling Systematic and Flexible Exploration of Fused-layer Dataflow Accelerators*. PhD thesis. Massachusetts Institute of Technology, Cambridge, MA.
- [15] Michael Gilbert, Tanner Andrusis, Vivienne Sze, and Joel S. Emer. 2026. The Turbo-Charged Mapper: Fast and Optimal Mapping for Accelerator Modeling and Evaluation. arXiv:2602.15172 [cs.AR] <https://arxiv.org/abs/2602.15172>
- [16] Michael Gilbert, Yannan Nellie Wu, Joel S. Emer, and Vivienne Sze. 2024. LoopTree: Exploring the Fused-Layer Dataflow Accelerator Design Space. *IEEE Transactions on Circuits and Systems for Artificial Intelligence* 1, 1 (2024), 97–111. <https://doi.org/10.1109/TCASAI.2024.3461716>
- [17] Koen Goetschalckx, Fengfeng Wu, and Marian Verhelst. 2023. DepFiN: A 12-nm Depth-First, High-Resolution CNN Processor for IO-Efficient Inference. *IEEE Journal of Solid-State Circuits* 58, 5 (2023), 1425–1435. <https://doi.org/10.1109/JSSC.2022.3210591>
- [18] Google. 2022. XLA: Optimizing Compiler for Machine Learning. <https://www.tensorflow.org/xla>
- [19] Kartik Hegde, Hadi Asghari-Moghaddam, Michael Pellauer, Neal Crago, Aamer Jaleel, Edgar Solomonik, Joel Emer, and Christopher W. Fletcher. 2019. ExTensor: An Accelerator for Sparse Tensor Algebra. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (Columbus, OH, USA) (MICRO '52)*. Association for Computing Machinery, New York, NY, USA, 319–333. <https://doi.org/10.1145/3352460.3358275>
- [20] Kartik Hegde, Po-An Tsai, Sitao Huang, Vikas Chandra, Angshuman Parashar, and Christopher W. Fletcher. 2021. Mind Mappings: Enabling Efficient Algorithm-Accelerator Mapping Space Search. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '21)*. IEEE.
- [21] Mark Horeni, Pooria Taheri, Po-An Tsai, Angshuman Parashar, Joel Emer, and Siddharth Joshi. 2022. Ruby: Improving Hardware Efficiency for Tensor Algebra Accelerators Through Imperfect Factorization. In *2022 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 254–266. <https://doi.org/10.1109/ISPASS55109.2022.00039>
- [22] Qijing Huang, Minwoo Kang, Grace Dinh, Thomas Norell, Aravind Kalaiah, James Demmel, John Wawrzynek, and Yakun Sophia Shao. 2021. CoSA: Scheduling by Constrained Optimization for Spatial Accelerators. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. 554–566. <https://doi.org/10.1109/ISCA52012.2021.00050>
- [23] Hongyang Jia, Hossein Valavi, Yinqi Tang, Jintao Zhang, and Naveen Verma. 2020. A Programmable Heterogeneous Microprocessor Based on Bit-Scalable In-Memory Computing. *IEEE Journal of Solid-State Circuits* 55, 9 (2020), 2609–2621. <https://doi.org/10.1109/JSSC.2020.2987714>
- [24] Norman P. Jouppi, Doe Hyun Yoon, Matthew Ashcraft, Mark Gottscho, Thomas B. Jablin, George Kurian, James Laudon, Sheng Li, Peter Ma, Xiaoyu Ma, Thomas Norrie, Nishant Patil, Sushma Prasad, Cliff Young, Zongwei Zhou, and David Patterson. 2021. Ten Lessons From Three Generations Shaped Google's TPUv4: Industrial Product. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. 1–14. <https://doi.org/10.1109/ISCA52012.2021.00010>
- [25] Sheng-Chun Kao and Tushar Krishna. 2020. GAMMA: Automating the HW Mapping of DNN Models on Accelerators via Genetic Algorithm. In *2020 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. 1–9.
- [26] Sheng-Chun Kao, Suvinay Subramanian, Gaurav Agrawal, Amir Yazdanbakhsh, and Tushar Krishna. 2021. FLAT: An Optimized Dataflow for Mitigating Attention Bottlenecks. <https://doi.org/10.48550/ARXIV.2107.06419>
- [27] Hyunjoon Kim, Taegyeon Yoo, Tony Tae-Hyoung Kim, and Bongjin Kim. 2021. Colonnade: A Reconfigurable SRAM-Based Digital Bit-Serial Compute-In-Memory Macro for Processing Neural Networks. *IEEE Journal of Solid-State Circuits* 56, 7 (2021), 2221–2233. <https://doi.org/10.1109/JSSC.2021.3061508>
- [28] Fredrik Kjolstad, Stephen Chou, David Lugato, Shoaib Kamil, and Saman Amarasinghe. 2017. Taco: A tool to generate tensor algebra kernels. In *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 943–948. <https://doi.org/10.1109/ASE.2017.8115709>
- [29] Hyounjun Kwon, Prasantha Chatarasi, Vivek Sarkar, Tushar Krishna, Michael Pellauer, and Angshuman Parashar. 2020. MAESTRO: A Data-Centric Approach to Understand Reuse, Performance, and Hardware Cost of DNN Mappings. *IEEE Micro* 40, 3 (2020), 20–29. <https://doi.org/10.1109/MM.2020.2985963>

- [30] Marco Laumanns, Lothar Thiele, Kalyanmoy Deb, and Eckart Zitzler. 2002. Combining Convergence and Diversity in Evolutionary Multiobjective Optimization. *Evolutionary Computation* 10, 3 (2002), 263–282. <https://doi.org/10.1162/10635602760234108>
- [31] L. Mei, K. Goetschalckx, A. Symons, and M. Verhelst. 2023. DeFiNES: Enabling Fast Exploration of the Depth-first Scheduling Space for DNN Accelerators through Analytical Modeling. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE Computer Society, Los Alamitos, CA, USA, 570–583. <https://doi.org/10.1109/HPCA56546.2023.10071098>
- [32] Linyan Mei, Pouya Houshmand, Vikram Jain, Sebastian Giraldo, and Marian Verhelst. 2021. ZigZag: Enlarging Joint Architecture-Mapping Design Space Exploration for DNN Accelerators. *IEEE Trans. Comput.* 70, 8 (2021), 1160–1174. <https://doi.org/10.1109/TC.2021.3059962>
- [33] Microsoft. 2025. Develop AI applications for Copilot+ PCs. <https://learn.microsoft.com/en-us/windows/ai/npu-devices/>. Accessed: 2026-04-06.
- [34] Nandeeeka Nayak, Toluwanimi O. Odemuyiwa, Shubham Ugare, Christopher W. Fletcher, Michael Pellauer, and Joel S. Emer. 2023. TeAAL: A Declarative Framework for Modeling Sparse Tensor Accelerators. *arXiv e-prints*, Article arXiv:2304.07931 (April 2023), arXiv:2304.07931 pages. <https://doi.org/10.48550/arXiv.2304.07931> [cs.AR]
- [35] Nandeeeka Nayak, Xinrui Wu, Toluwanimi O. Odemuyiwa, Michael Pellauer, Joel S. Emer, and Christopher W. Fletcher. 2024. FuseMax: Leveraging Extended Einsums to Optimize Attention Accelerator Design. In *Proceedings of the 57th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '24)*. Association for Computing Machinery, New York, NY, USA.
- [36] Toluwanimi O. Odemuyiwa, Joel S. Emer, and John D. Owens. 2024. The EDGE Language: Extended General Einsums for Graph Algorithms. *arXiv e-prints*, Article arXiv:2404.11591 (April 2024), arXiv:2404.11591 pages. <https://doi.org/10.48550/arXiv.2404.11591> [cs.DS]
- [37] Angshuman Parashar, Priyanka Raina, Yakun Sophia Shao, Yu-Hsin Chen, Victor A. Ying, Anurag Mukkara, Rangharajan Venkatesan, Bruce Khailany, Stephen W. Keckler, and Joel Emer. 2019. Timeloop: A Systematic Approach to DNN Accelerator Evaluation. In *2019 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 304–315. <https://doi.org/10.1109/ISPASS.2019.00042>
- [38] Ali Shafiee, Anirban Nag, Naveen Muralimanohar, Rajeev Balasubramanian, John Paul Strachan, Miao Hu, R. Stanley Williams, and Vivek Srikumar. 2016. ISAAC: A Convolutional Neural Network Accelerator with In-Situ Analog Arithmetic in Crossbars. In *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*. 14–26. <https://doi.org/10.1109/ISCA.2016.12>
- [39] Kyle Shiflett, Avinash Karanth, Razvan Bunescu, and Ahmed Louri. 2021. Albireo: energy-efficient acceleration of convolutional neural networks via silicon photonics. In *Proceedings of the 48th Annual International Symposium on Computer Architecture (Virtual Event, Spain) (ISCA '21)*. IEEE Press, 860–873. <https://doi.org/10.1109/ISCA52012.2021.00072>
- [40] Mahmut E. Sinangil, Burak Erbagci, Rawan Naous, Kerem Akarvardar, Dar Sun, Win-San Khwa, Hung-Jen Liao, Yih Wang, and Jonathan Chang. 2021. A 7-nm Compute-in-Memory SRAM Macro Supporting Multi-Bit Input, Weight and Output and Achieving 351 TOPS/W and 372.4 GOPS. *IEEE Journal of Solid-State Circuits* 56, 1 (2021), 188–198. <https://doi.org/10.1109/JSSC.2020.3031290>
- [41] Vivienne Sze, Yu-Hsin Chen, Tien-Ju Yang, and Joel S. Emer. 2020. Efficient Processing of Deep Neural Networks. *Synthesis Lectures on Computer Architecture* 15, 2 (2020), 1–341. <https://doi.org/10.2200/S01004ED1V01Y202004CAC050> arXiv:https://doi.org/10.2200/S01004ED1V01Y202004CAC050
- [42] Luc Waeijen, Savvas Sioutas, Maurice Peemen, Menno Lindwer, and Henk Corporaal. 2021. ConvFusion: A Model for Layer Fusion in Convolutional Neural Networks. *IEEE Access* 9 (2021), 168245–168267. <https://doi.org/10.1109/ACCESS.2021.3134930>
- [43] Weier Wan, Rajkumar Kubendran, S. Burc Eryilmaz, Wenqiang Zhang, Yan Liao, Dabin Wu, Stephen Deiss, Bin Gao, Priyanka Raina, Siddharth Joshi, Huaqiang Wu, Gert Cauwenberghs, and H.-S. Philip Wong. 2020. 33.1 A 74 TMACS/W CMOS-RRAM Neurosynaptic Core with Dynamically Reconfigurable Dataflow and In-situ Transposable Weights for Probabilistic Graphical Models. In *2020 IEEE International Solid-State Circuits Conference - (ISSCC)*. 498–500. <https://doi.org/10.1109/ISSCC19947.2020.9062979>
- [44] Weier Wan, Rajkumar Kubendran, Clemens Schaefer, Sukru Burc Eryilmaz, Wenqiang Zhang, Dabin Wu, Stephen Deiss, Priyanka Raina, He Qian, Bin Gao, Siddharth Joshi, Huaqiang Wu, H.-S. Philip Wong, and Gert Cauwenberghs. 2022. A compute-in-memory chip based on resistive random-access memory. *Nature* 608, 7923 (Aug. 2022), 504–512. <https://doi.org/10.1038/s41586-022-04992-8> Number: 7923 Publisher: Nature Publishing Group.
- [45] Hechen Wang, Renzhi Liu, Richard Dorrance, Deepak Dasalukunte, Dan Lake, and Brent Carlton. 2023. A Charge Domain SRAM Compute-in-Memory Macro With C-2C Ladder-Based 8-Bit MAC Unit in 22-nm FinFET Process for Edge Inference. *IEEE Journal of Solid-State Circuits* 58, 4 (2023), 1037–1050. <https://doi.org/10.1109/JSSC.2022.3232601>
- [46] Hechen Wang, Renzhi Liu, Richard Dorrance, Deepak Dasalukunte, Xiaosen Liu, Dan Lake, Brent Carlton, and May Wu. 2022. A 32.2 TOPS/W SRAM Compute-in-Memory Macro Employing a Linear 8-bit C-2C Ladder for Charge Domain Computation in 22nm for Edge Inference. In *2022 IEEE Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits)*. 36–37. <https://doi.org/10.1109/VLSITechnologyandCirc46769.2022.9830322>
- [47] Yuchen Wei, Jingwei Cai, Zuotong Wu, Sen Peng, and Kaisheng Ma. 2023. SET Artifacts. <https://doi.org/10.5281/zenodo.7751328>
- [48] Yannan Nellie Wu, Joel S. Emer, and Vivienne Sze. 2019. Accelergy: An Architecture-Level Energy Estimation Methodology for Accelerator Designs. In *2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. 1–8. <https://doi.org/10.1109/ICCAD45719.2019.8942149>
- [49] Linxuan Zhang, J. Nelson Amaral, and Di Niu. 2025. TransFusion: End-to-End Transformer Acceleration via Graph Fusion and Pipelining. Association for Computing Machinery, New York, NY, USA, 1491–1504. <https://doi.org/10.1145/3725843.3756105>
- [50] Size Zheng, Siyuan Chen, Siyuan Gao, Liancheng Jia, Guangyu Sun, Runsheng Wang, and Yun Liang. 2023. TileFlow: A Framework for Modeling Fusion Dataflow via Tree-Based Analysis. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture (<conf-loc>, <city>Toronto</city>, <state>ON</state>, <country>Canada</country>, </conf-loc>)* (MICRO '23). Association for Computing Machinery, New York, NY, USA, 1271–1288. <https://doi.org/10.1145/3613424.3623792>
- [51] Zhizhen Zhong, Mingran Yang, Jay Lang, Christian Williams, Liam Kronman, Alexander Sludds, Homa Esfahanizadeh, Dirk Englund, and Manya Ghobadi. 2023. Lighting: A Reconfigurable Photonic-Electronic SmartNIC for Fast and Energy-Efficient Inference. In *Proceedings of the ACM SIGCOMM 2023 Conference* (New York, NY, USA) (ACM SIGCOMM '23). Association for Computing Machinery, New York, NY, USA, 452–472. <https://doi.org/10.1145/3603269.3604821>